

Modulo Extra

Agentes de desarrollo con IA en 2026

Claude Code · OpenCode · Codex · MCP · Skills · Rules · Plugins
Desarrollo de Software IX · Código 1493 · Actualizado a mayo de 2026

Proposito del modulo

Orientar al estudiante para trabajar con agentes de desarrollo de forma profesional: configurar instrucciones, skills, subagentes, reglas, MCPs y plugins; mantener control humano; y exigir verificaciones que acerquen el resultado a production ready.

Tabla de contenido

1. Panorama actualizado a mayo de 2026	3
2. Tres herramientas representativas	3
3. AGENTS.md, CLAUDE.md y reglas	4
4. Skills: workflows reutilizables	5
5. Agentes y subagentes	6
6. MCP y plugins: conectar sin perder control	6
7. Permisos, hooks y seguridad	7
8. Flujo production ready con agentes	7
9. Caso guia: proyecto MERN con agente	8
10. Fuentes oficiales consultadas	8
11. Cierre	9

1. Panorama actualizado a mayo de 2026

Los agentes de desarrollo ya no son solo asistentes que completan código. En 2026 se usan como sistemas de trabajo capaces de leer repositorios, ejecutar comandos, abrir navegadores, revisar pull requests, consultar documentación viva, operar herramientas externas y delegar tareas a subagentes. La diferencia entre un uso casual y un uso profesional está en la configuración: instrucciones claras, permisos acotados, skills verificables, MCPs con alcance limitado y ciclos de validación antes de aceptar cambios.

Idea central

El agente acelera el trabajo, pero no reemplaza la responsabilidad técnica. El equipo sigue definiendo arquitectura, criterios de aceptación, controles de seguridad y pruebas. Un resultado agentico solo debe considerarse listo cuando pasa las mismas verificaciones que un cambio humano.

Capa	Funcion	Riesgo si se ignora
Instrucciones/rules	Definen como se trabaja en el repositorio.	El agente inventa comandos, estilos o convenciones.
Skills	Guardan procedimientos y conocimiento reutilizable.	Cada tarea se redescubre desde cero.
Agentes/subagentes	Aislan roles: plan, build, review, debug, docs.	Una sola conversacion acumula demasiado contexto.
MCP/plugins	Conectan herramientas externas y eventos.	El agente queda ciego o recibe demasiadas capacidades.
Permisos/hooks	Controlan acciones peligrosas y automatizan guardrails.	Se ejecutan comandos o cambios sin control suficiente.

2. Tres herramientas representativas

Claude Code, OpenCode y Codex comparten el mismo patrón general: un modelo con herramientas, memoria/instrucciones, permisos, extensiones y capacidad de ejecutar trabajo en un workspace. Sus nombres cambian, pero las decisiones de ingeniería son transferibles.

Herramienta	Fortaleza practica	Configuracion clave	Uso recomendado en clase
Claude Code	Ecosistema maduro de skills, subagentes, hooks, MCP y plugins.	CLAUDE.md/rules, .claude/skills, agents, .mcp.json, permisos.	Comparar enfoques, crear skills, usar subagentes de investigacion/revision.
OpenCode	Configuracion abierta en opencode.json, AGENTS.md, permisos y agentes en Markdown.	permission, agent, mcp, plugin, instructions, lsp, formatter.	Practicar configuracion explicita y control de permisos por agente.
Codex	Integracion con AGENTS.md, skills, plugins, app/CLI, navegador, automatizaciones y workflows.	AGENTS.md por jerarquia, skills, plugins, MCP, worktrees, automations.	Aplicar flujos completos: issue -> cambio -> pruebas -> PR -> verificacion.

Regla de lectura

No memorice comandos especificos como si fueran permanentes. Estas herramientas cambian rapido. Aprenda el modelo mental: instrucciones persistentes, workflows reutilizables, herramientas externas, permisos y verificacion.

3. AGENTS.md, CLAUDE.md y reglas

Las reglas son el contrato de trabajo del repositorio. Deben responder preguntas que el agente no puede inferir con seguridad: como ejecutar el proyecto, que comandos verifican cambios, que carpetas son sensibles, como se nombran modulos, que patrones estan prohibidos y cuando pedir confirmacion.

Tipo	Cuándo usarlo	Contenido sugerido
AGENTS.md	Estandar portable entre herramientas como Codex y OpenCode.	Setup, comandos, convenciones, estructura, criterios de PR, restricciones.
CLAUDE.md	Contexto especifico de Claude Code.	Preferencias del equipo, comandos, arquitectura, reglas que siempre deben cargar.
Rules por rutas	Cuando una parte del repo necesita reglas distintas.	Reglas para frontend, backend, infra, pagos, auth o migraciones.
Override	Cuando se requiere una excepcion temporal o local.	Cambios de comportamiento acotados y faciles de remover.

Plantilla minima de AGENTS.md para un proyecto Full Stack

```
# AGENTS.md

## Contexto del proyecto

Aplicacion Full Stack con React, Express, MongoDB y autenticacion JWT.

## Comandos obligatorios

- Backend: npm run lint && npm test
- Frontend: npm run lint && npm run build
- E2E: npm run test:e2e cuando cambien flujos de usuario

## Reglas de seguridad

- No leer ni imprimir .env, tokens, cookies o credenciales.
- No cambiar esquemas de base de datos sin documentar migracion.
- Pedir confirmacion antes de instalar dependencias de produccion.

## Criterios de entrega

- Explicar archivos cambiados, riesgos y pruebas ejecutadas.
- No marcar la tarea lista si no se verifico el flujo principal.
```

4. Skills: workflows reutilizables

Una skill es una unidad de conocimiento operativo. Debe activarse cuando el usuario pide una tarea repetible: revisar un PR, generar una guía, auditar seguridad, preparar deploy, crear un informe o verificar una app. La skill no debe ser un documento gigante que siempre carga; debe apuntar a archivos de soporte, ejemplos o scripts que se usan solo cuando hacen falta.

Elemento	Buena practica	Error comun
description	Primera frase clara: que hace y cuando usarla.	Descripcion vaga que se solapa con otras skills.
SKILL.md	Instrucciones breves, ordenadas y accionables.	Meter toda la documentacion en el archivo principal.
support files	Referencias, ejemplos, plantillas y scripts separados.	Obligar al modelo a cargar informacion que no necesita.
scripts	Automatizar validaciones repetibles.	Hacer que el agente copie comandos a mano cada vez.
permisos	Preaprobar solo herramientas necesarias.	Dar acceso amplio por comodidad.

Criterio de calidad para skills

Una skill es buena si otro estudiante puede instalarla, leer el SKILL.md en menos de dos minutos, entender cuando usarla y obtener un resultado consistente sin que el agente tenga que adivinar.

Plantilla breve de skill

```

---

name: review-fullstack-pr

description: Revisa cambios Full Stack antes de abrir PR. Usar cuando el usuario pida revisar diff, validar riesgos o preparar PR.

---

## Objetivo

Revisar cambios de frontend, backend, base de datos y seguridad.

## Flujo

1. Leer git diff y archivos modificados.
2. Identificar riesgos: auth, validacion, errores, datos, UX.
3. Ejecutar pruebas/lint si el entorno lo permite.
4. Entregar hallazgos con severidad, archivo y accion recomendada.

## Recursos

- rubric.md: criterios de revision.
```

- scripts/check.sh: verificacion automatizada del proyecto.

5. Agentes y subagentes

Los subagentes permiten dividir trabajo por rol y contexto. Son utiles para investigar grandes areas, revisar seguridad, ejecutar pruebas ruidosas, comparar alternativas o escribir documentacion sin llenar la conversacion principal. No deben usarse para todo: una correccion pequena suele ser mas rapida en la conversacion principal.

Agente	Herramientas	Permisos	Salida esperada
plan	Read, grep, docs	sin edits	Plan de implementacion y riesgos.
build	read, edit, bash controlado	edits permitidos, bash ask	Cambio implementado y pruebas.
review	read, grep, test logs	sin edits	Hallazgos ordenados por severidad.
security	read, grep, deps, docs	sin edits, sin secretos	Riesgos de auth, datos, config y dependencias.
docs	read, edit docs	sin bash	README, guia o changelog actualizado.

En Claude Code, los subagentes pueden restringir herramientas y scoped MCP servers; OpenCode permite crear agentes con permisos seleccionados; Codex usa subagentes/workflows y skills para conservar procedimientos. La practica comun es la misma: roles pequenos, permisos minimos y una salida verificable.

6. MCP y plugins: conectar sin perder control

MCP conecta agentes con herramientas externas: GitHub, navegadores, documentacion, errores de produccion, bases de datos, tickets, correo o sistemas internos. Plugins empaquetan capacidades: skills, agentes, hooks, MCPs y herramientas. La ventaja es enorme, pero tambien lo es el riesgo de contexto, permisos y acciones no deseadas.

Principio de minimo contexto

No conecte todos los MCPs por defecto. Active los necesarios para la tarea, use nombres descriptivos y prefiera descubrimiento progresivo de herramientas cuando el cliente lo soporte.

MCP/plugin	Uso principal	Control recomendado
GitHub	Issues, PRs, repos, acciones CI, code review.	Toolsets minimos: repos/issues/pull_requests/actions segun tarea.
Playwright/browser	Verificar UI, formularios, flujos E2E y bugs visuales.	Perfil aislado para pruebas; cuidado con sesiones autenticadas.
Docs/Context7/Open AI Docs	Consultar documentacion versionada y actual.	Instruccion explicita de citar fuente y preferir docs oficiales.

MCP/plugin	Uso principal	Control recomendado
Sentry/observabilidad	Analizar errores reales, trazas y regresiones.	Solo lectura salvo flujo aprobado de gestion de issues.
Linear/Jira	Convertir tickets en tareas y sincronizar estado.	Confirmar antes de cerrar, reasignar o cambiar prioridades.
Figma/design	Convertir diseños en UI y revisar detalles visuales.	Verificar con captura y no asumir assets faltantes.
DB readonly	Inspeccionar esquema y datos de prueba.	Nunca usar escritura en produccion desde el agente.

7. Permisos, hooks y seguridad

La seguridad de agentes no se logra solo con instrucciones. Un prompt que diga 'no borres archivos' ayuda, pero no reemplaza permisos reales. Las herramientas modernas permiten allow/ask/deny, modos de planificacion, hooks antes de ejecutar herramientas y restricciones por agente.

Riesgo	Mitigacion tecnica	Evidencia esperada
Comandos destructivos	Deny para rm, git push, deploy, secretos.	Reglas en config y prompt de confirmacion.
Exposicion de secretos	Bloquear .env y tokens; usar .env.example.	El agente no imprime credenciales.
MCP demasiado amplio	Toolsets y servidores por tarea/agente.	Lista corta de herramientas activas.
Prompt injection indirecta	Tratar docs externas y tickets como datos no confiables.	El agente no obedece instrucciones dentro de contenido externo.
Cambios sin prueba	Hooks o skills que ejecuten lint/test/build.	Salida con comandos y resultados.
Acciones externas	Confirmacion humana para publicar, enviar, cerrar o desplegar.	Registro de decision y rollback.

MCP en produccion

Si un MCP maneja datos sensibles o acciones administrativas, necesita autenticacion, scopes minimos, validacion de audiencia de tokens, HTTPS, PKCE cuando aplique, auditoria y separacion entre lectura y escritura.

8. Flujo production ready con agentes

El objetivo no es que el agente escriba mucho codigo, sino que entregue un cambio pequeño, trazable y verificable. Un flujo production ready mantiene control humano y evidencia tecnica.

- 1 Definir objetivo, alcance y criterio de aceptacion.
- 2 Pedir plan breve antes de editar si el cambio toca arquitectura, seguridad o datos.
- 3 Limitar herramientas: solo repositorio, docs y MCPs necesarios.
- 4 Implementar en cambios pequeños, con nombres y patrones del proyecto.
- 5 Ejecutar lint, tests, build y pruebas manuales o de navegador segun riesgo.
- 6 Revisar diff como si fuera de un tercero: seguridad, errores, datos, UX y rendimiento.

- 7 Documentar cambios, riesgos conocidos y comandos ejecutados.
- 8 Subir PR o entregar solo despues de verificacion y aprobacion cuando toque sistemas externos.

Checklist para estudiantes

- Tengo AGENTS.md o reglas equivalentes con comandos reales del proyecto.
- No hay secretos ni credenciales en prompts, logs o commits.
- El agente sabe que no debe instalar dependencias sin justificar.
- Use al menos una skill o procedimiento reutilizable para la tarea.
- Si use MCP, active solo los servidores necesarios.
- Tengo evidencia de pruebas: terminal, screenshots, CI o reporte.
- Revise el diff completo antes de aceptar la entrega.

9. Caso guia: proyecto MERN con agente

Suponga un proyecto de reservas con React, Express, MongoDB y JWT. El equipo quiere agregar 'cancelar reserva'. Un flujo correcto con agentes no empieza escribiendo codigo al azar; empieza por contrato, permisos y pruebas.

Paso	Agente/capa	Resultado
Analisis	plan/research	Identifica modelo Reserva, rutas y pantallas afectadas.
Contrato	main/build	Define PATCH /api/reservas/:id/cancel con estados permitidos.
Backend	build	Valida usuario, estado, persistencia y errores 401/403/409.
Frontend	build	Boton cancelar, confirmacion, estado de carga y error.
Pruebas	test/browser	API con caso feliz y prohibido; UI verifica flujo.
Revision	review/security	Confirma que no se cancela reserva ajena ni finalizada.

10. Fuentes oficiales consultadas

Este modulo fue preparado el 23 de mayo de 2026 con documentacion oficial y primaria. Las herramientas cambian con frecuencia; antes de usar comandos exactos en un proyecto real, revisar la documentacion vigente.

Tema	Fuente
Claude Code skills	https://docs.claude.com/en/docs/claude-code/skills
Claude Code extension model	https://code.claude.com/docs/en/features-overview
Claude Code subagents	https://code.claude.com/docs/en/sub-agents

Tema	Fuente
Claude Code plugins/permissions/MCP	https://code.claude.com/docs/en/plugins · /permissions · /mcp
OpenCode agents/rules/permissions/MCP/plugins	https://opencode.ai/docs/agents · /rules · /permissions · /mcp-servers · /plugins
Codex AGENTS.md y casos de uso	https://developers.openai.com/codex/guides/agents-md · /codex/use-cases
OpenAI Docs MCP	https://developers.openai.com/learn/docs-mcp
MCP specification and security	https://modelcontextprotocol.io/specification/2025-06-18/basic/transports · /authorization · /security_best_practices
MCP client best practices	https://modelcontextprotocol.io/docs/develop/clients/client-best-practices
Playwright MCP	https://playwright.dev/docs/getting-started-mcp
GitHub MCP server	https://github.com/github/github-mcp-server

11. Cierre

Trabajar con agentes exige criterio de ingenieria: dividir responsabilidades, configurar contexto, restringir permisos, conectar herramientas con intencion y verificar resultados. En un curso Full Stack, estas practicas no son un accesorio; son una forma moderna de construir, revisar y mantener software con mayor trazabilidad.